

Ejercicio 2 - Parte 2 (Resuelva en C)

Enunciado

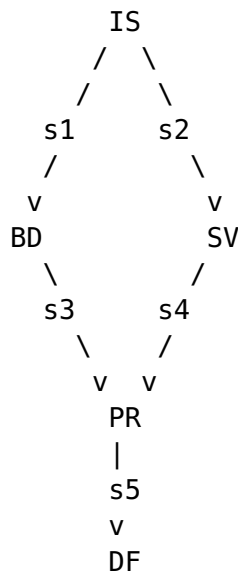
Un equipo de desarrollo debe desplegar una aplicación siguiendo un conjunto de tareas con dependencias:

- IS (Inicializar sistema): sin previas
- BD (Configurar base de datos): requiere IS
- SV (Configurar servidor): requiere IS
- PR (Ejecutar pruebas): requiere BD y SV
- DF (Deploy final): requiere PR

Se pide:

1. Dibujar el grafo de precedencias.
2. Explicar cómo se inicializarían los semáforos.
3. Justificar por qué dicha inicialización es correcta.

1. Grafo de precedencias



El grafo tiene cinco nodos y cinco aristas, sin ciclos. IS es la única tarea sin predecesores y DF la única sin sucesores. BD y SV pueden ejecutarse en paralelo y vuelven a sincronizarse en PR. Como se usa un semáforo por arista, el problema requiere cinco semáforos.

2. Inicialización de los semáforos

Para sincronizar tareas con precedencia en un esquema cobegin/coend se usa un semáforo por cada arista del grafo, inicializado en cero. Cada tarea hace P() sobre los semáforos de sus aristas entrantes y V() sobre los de sus aristas salientes.

s1, s2, s3, s4, s5: semáforos

```
init(s1, 0)    // arista IS -> BD
init(s2, 0)    // arista IS -> SV
init(s3, 0)    // arista BD -> PR
init(s4, 0)    // arista SV -> PR
init(s5, 0)    // arista PR -> DF
```

Pseudocódigo del programa concurrente:

```
cobegin
  begin IS; V(s1); V(s2); end
  begin P(s1); BD; V(s3); end
  begin P(s2); SV; V(s4); end
  begin P(s3); P(s4); PR; V(s5); end
  begin P(s5); DF; end
coend
```

Resumen de operaciones por tarea:

Tarea	P() entrantes	V() salientes
IS	-	V(s1), V(s2)
BD	P(s1)	V(s3)
SV	P(s2)	V(s4)
PR	P(s3), P(s4)	V(s5)
DF	P(s5)	-

3. Justificación

Que todos los semáforos arranquen en cero implica que cualquier P() ejecutado antes de su V() correspondiente bloquea al proceso que lo invoca. Eso es lo que fuerza el orden definido por el grafo.

IS no tiene predecesores y arranca sin bloquearse. Cuando termina hace $V(s1)$ y $V(s2)$, lo que libera a BD y SV. Como ninguna de las dos depende de la otra, pueden correr en paralelo, que es justamente lo que el grafo permite.

PR depende de BD y SV a la vez. Por eso ejecuta $P(s3)$ y $P(s4)$ antes de empezar su trabajo. El orden entre los dos $P()$ no importa: PR no avanza hasta que se hayan hecho los dos $V()$ correspondientes. De esta forma se modela la dependencia conjunta sobre ambas tareas.

DF solo depende de PR. El $P(s5)$ al inicio asegura que no arranque antes de que PR haya terminado.

No se produce deadlock porque el grafo es acíclico y cada $P()$ está pareado con un $V()$ emitido por alguna tarea que efectivamente se ejecuta. Ningún proceso queda esperando una señal que no vaya a llegar.

Sobre la elección de un semáforo por arista y no uno por nodo: cuando un nodo tiene más de un sucesor, un solo $V()$ sobre un semáforo contador libera apenas un proceso bloqueado en $P()$. Si IS hiciera un único $V()$ sobre un semáforo compartido, solamente BD o SV se desbloquearía y la otra rama quedaría esperando. Con un semáforo por arista, IS hace $V(s1)$ y $V(s2)$ y cada rama recibe su propia señal.